

GS 2° Semestre – Python

Sistema de Monitoramento da Missão Espacial Orion-X

Nome: Matheus Costa Cutrim

RM: 568087

Turma: CCPR

Professor: Allan Roberto

• Introdução

A exploração espacial moderna depende da coleta e análise constante de dados para garantir a segurança das missões, o funcionamento adequado dos equipamentos e a tomada de decisões em tempo real. Sensores instalados em naves e módulos espaciais geram grandes volumes de informações relacionadas à temperatura, níveis de energia, comunicação, radiação e diversas outras variáveis essenciais para o sucesso das operações.

Nesse contexto, a tecnologia da informação desempenha um papel fundamental, permitindo automatizar processos de monitoramento, armazenar dados de forma organizada e transformar informações brutas em relatórios estratégicos para apoio às equipes responsáveis pela missão. Ferramentas de análise de dados, integração com sistemas externos e geração automática de relatórios são amplamente utilizadas na indústria espacial para aumentar a eficiência e reduzir riscos operacionais.

Com base nesse cenário, foi desenvolvido o **Sistema de Monitoramento da Missão Espacial Orion-X**, uma aplicação em Python criada para simular o acompanhamento de uma missão espacial fictícia. O sistema utiliza estruturas de dados, manipulação de arquivos Excel, análise com Pandas, integração com API Web e automação de relatórios, permitindo coletar, processar e analisar informações relevantes da missão. Dessa forma, o projeto demonstra na prática como os conceitos estudados na disciplina podem ser aplicados para resolver problemas relacionados ao monitoramento inteligente e à análise de dados no setor aeroespacial.

• Justificativa

A indústria espacial moderna depende cada vez mais de sistemas computacionais capazes de monitorar, processar e analisar grandes quantidades de dados em tempo real. Durante uma missão espacial, informações relacionadas à temperatura, níveis de energia, comunicação, radiação e funcionamento dos módulos precisam ser acompanhadas constantemente para garantir a segurança da nave e o sucesso da operação.

Nesse cenário, a utilização de ferramentas de análise de dados e automação torna-se essencial, pois permite identificar rapidamente possíveis falhas, prever situações de risco e auxiliar na tomada de decisões. Além disso, a integração entre diferentes fontes de informação possibilita uma visão mais completa do estado da missão, aumentando a eficiência do monitoramento.

O desenvolvimento deste projeto foi motivado pela necessidade de simular esse ambiente de controle e análise, aplicando na prática os conceitos estudados na disciplina. Através da utilização de Python, estruturas de dados, manipulação de arquivos Excel, análise com Pandas e integração com APIs Web, foi possível construir uma solução capaz de organizar informações, gerar análises automáticas e produzir relatórios de forma eficiente.

Dessa forma, o projeto contribui para o entendimento de como tecnologias amplamente utilizadas no mercado podem ser aplicadas em contextos da indústria espacial, demonstrando a importância da programação e da análise de dados para o monitoramento inteligente de sistemas complexos.

• Objetivos

O objetivo geral do projeto é desenvolver uma aplicação em Python capaz de monitorar, processar e analisar dados de uma missão espacial fictícia, utilizando técnicas de automação, manipulação de arquivos Excel, análise de dados e integração com serviços externos, com o objetivo de gerar informações estratégicas e relatórios automatizados para apoio ao monitoramento da missão.

Assim como também:

- Aplicar estruturas de dados como tuplas, dicionários e laços de repetição para armazenar e organizar informações relacionadas à missão espacial.
- Realizar a leitura e manipulação de arquivos Excel utilizando a biblioteca Pandas, permitindo o tratamento e a atualização dos dados monitorados.
- Desenvolver análises automáticas dos dados coletados por meio de funções estatísticas e filtros condicionais, facilitando a identificação de situações normais, de atenção e críticas.

- Criar novas colunas e indicadores capazes de fornecer informações complementares sobre o desempenho e a estabilidade da missão.
- Integrar a aplicação a uma API Web pública utilizando a biblioteca Requests, possibilitando a obtenção de dados externos em formato JSON e ampliando a capacidade de monitoramento do sistema.
- Implementar mecanismos de tratamento de erros para garantir maior confiabilidade durante a comunicação com serviços externos.
- Automatizar a consolidação das informações processadas e a geração de relatórios organizados em formato Excel.
- Produzir um relatório final contendo dados analisados, classificações de risco, resumos estatísticos, alertas operacionais e informações complementares obtidas pela API.
- Demonstrar, na prática, a aplicação dos conceitos estudados na disciplina em um cenário inspirado nos desafios reais da indústria espacial.

• Código Fonte

Arquivo do código:



sistema_missao_orion_x.py

Link do código no colab:

https://colab.research.google.com/drive/1wTLHYjENj5p81a_cfKO8a1lu6dVDBZUi?usp=drive_link

Explicação:

O código desenvolvido corresponde ao Sistema de Monitoramento da Missão Espacial Orion-X, uma aplicação em Python criada para simular o acompanhamento de dados de uma missão espacial fictícia. O programa utiliza estruturas de dados, leitura e manipulação de arquivos Excel, análise com Pandas, integração com API Web e geração automática de relatório final.

1. Importação das bibliotecas

No início do código, são importadas as bibliotecas necessárias:

```
import pandas as pd
```

```
import requests
```

```
from datetime import datetime
```

A biblioteca pandas é utilizada para ler, manipular e analisar os dados da planilha Excel. A biblioteca requests permite consumir uma API pública pela internet. Já datetime é usada para registrar a data e hora em que o relatório final foi gerado.

2. Definição dos arquivos

O código define dois arquivos principais:

```
ARQUIVO_ENTRADA = "base_missao_orion_x.xlsx"
```

```
ARQUIVO_SAIDA = "relatorio_final_orion_x.xlsx"
```

O primeiro representa a base de dados fictícia da missão. O segundo é o relatório automatizado que será criado após o processamento dos dados.

3. Função: `exibir_estruturas_missao()`

Essa função demonstra o uso de **tuplas, dicionários e laços de repetição**, conforme solicitado na atividade.

A tupla `coordenadas_nave` armazena latitude, longitude e altitude simulada da nave:

```
coordenadas_nave = (-23.5505, -46.6333, 408.0)
```

As informações dos sensores são armazenadas em um dicionário chamado `sensores`, no qual cada sensor possui código, tipo, módulo e unidade de medida. Também existe o dicionário `modulos`, que identifica os módulos da nave.

Depois disso, o código usa laços `for` para percorrer os sensores e módulos, imprimindo tudo de forma organizada no terminal.

4. Função: `classificar_registro()`

Essa função recebe cada linha da planilha e classifica o risco da missão como:

- Normal;
- Atencao;
- Critico.

A classificação é feita com base em condições envolvendo temperatura, energia, oxigênio, radiação e comunicação.

Por exemplo:

```
if temperatura >= 38 or energia < 30 or oxigenio < 70 or radiacao > 1.20 or comunicacao < 60:
```

```
    return "Critico"
```

Isso significa que, se qualquer uma dessas variáveis estiver em nível perigoso, o registro será classificado como crítico.

Caso os valores estejam fora do ideal, mas ainda não em nível extremo, o sistema retorna Atenção. Se tudo estiver dentro dos limites seguros, retorna Normal.

5. Função: gerar_analise_automatica()

Essa função gera uma explicação automática para cada registro analisado.

Ela cria uma lista chamada problemas e adiciona mensagens conforme os dados encontrados. Por exemplo, se a temperatura estiver alta, o sistema adiciona “temperatura elevada”. Se a energia estiver baixa, adiciona “energia baixa”.

Caso nenhum problema seja encontrado, o sistema retorna:

"Parametros dentro da faixa segura."

Se existirem problemas, retorna uma frase de alerta, como:

Alerta: temperatura elevada, energia baixa.

Essa parte deixa o relatório mais interpretativo, pois não mostra apenas números, mas também uma explicação textual sobre a situação da missão.

6. Função: buscar_dados_api()

Essa função faz a integração com uma **API Web pública**, usando a biblioteca requests.

A API utilizada é a Open-Meteo, que fornece dados climáticos. No código, foram usadas coordenadas relacionadas ao Centro Espacial Kennedy:

```
parametros = {  
    "latitude": 28.5721,  
    "longitude": -80.6480,  
    "current_weather": True,  
}
```

O código tenta acessar a API dentro de um bloco try. Se a conexão funcionar, os dados são convertidos para JSON:

```
dados_json = resposta.json()
```

Depois, o programa extrai informações como temperatura externa, velocidade do vento, direção do vento e horário da coleta.

Se ocorrer erro na conexão, o bloco except evita que o sistema pare de funcionar. Em vez disso, ele exibe uma mensagem de erro e retorna valores vazios.

Essa etapa atende ao requisito de uso de:

- requests;

- try/except;
- leitura de JSON;
- apresentação dos dados coletados.

7. Função: `processar_dados_missao()`

Essa é a função principal do processamento dos dados.

Primeiro, o código lê a planilha Excel:

```
df = pd.read_excel(ARQUIVO_ENTRADA, sheet_name="Dados_Missao")
```

Depois, exibe uma confirmação:

```
print("Arquivo lido com sucesso!")
```

Em seguida, o programa executa comandos importantes do Pandas:

```
print(df.head())
```

```
print(df.tail())
```

```
df.info()
```

```
print(df.describe())
```

- `head()` mostra as primeiras linhas da planilha.
- O `.tail()` mostra as últimas linhas.
- O `.info()` mostra os tipos de dados e se existem valores vazios.
- O `.describe()` gera um resumo estatístico com média, mínimo, máximo e desvio padrão.

8. Criação de novas colunas

Depois da leitura e análise inicial, o sistema cria novas colunas no DataFrame:

```
df["classificacao_risco"] = df.apply(classificar_registro, axis=1)
```

```
df["analise_automatica"] = df.apply(gerar_analise_automatica, axis=1)
```

```
df["energia_consumida_pct"] = 100 - df["energia_pct"]
```

A coluna `classificacao_risco` mostra se o registro está normal, em atenção ou crítico.

A coluna `analise_automatica` apresenta uma explicação textual da situação.

A coluna `energia_consumida_pct` calcula quanto da energia já foi consumida.

Também é criado o `indice_estabilidade`, que combina energia, oxigênio, comunicação, radiação e temperatura para gerar um indicador geral da estabilidade da missão.

```
df["indice_estabilidade"] = (
    (df["energia_pct"] * 0.30)
    + (df["oxigenio_pct"] * 0.30)
```

```
+ (df["comunicacao_pct"] * 0.25)
- (df["radiacao_msv"] * 10)
- (df["temperatura_c"] * 0.15)
).round(2)
```

Quanto maior esse índice, melhor a estabilidade do registro monitorado.

9. Filtros condicionais

O código também aplica filtros condicionais para separar registros críticos e registros em atenção:

```
registros_criticos = df[df["classificacao_risco"] == "Critico"]
registros_atencao = df[df["classificacao_risco"] == "Atencao"]
```

Esses filtros permitem identificar rapidamente quais partes da missão precisam de atenção imediata.

Depois, o programa imprime esses registros no terminal.

10. Resumos da missão

O sistema gera dois resumos principais.

O primeiro é o resumo estatístico:

```
resumo_estatistico = df.describe()
```

Ele apresenta medidas como média, mínimo, máximo e desvio padrão dos dados numéricos.

O segundo é o resumo por classificação:

```
resumo_classificacao = df["classificacao_risco"].value_counts().reset_index()
```

Esse resumo mostra quantos registros estão em cada categoria: Normal, Atenção ou Crítico.

Além disso, o código cria um `resumo_geral`, com informações consolidadas, como total de registros, quantidade de alertas, médias dos sensores e data de geração do relatório.

11. Exportação do relatório final

Ao final, o código gera automaticamente um novo arquivo Excel:

```
with pd.ExcelWriter(ARQUIVO_SAIDA, engine="openpyxl") as writer:
```

Esse relatório contém várias abas:

- `Dados_Analisados`;
- `Resumo_Estatistico`;
- `Resumo_Classificacao`;

- Alertas_Criticos;
- Alertas_Atencao;
- Dados_API;
- Resumo_Geral.

Cada aba organiza uma parte diferente da análise. Isso torna o resultado final mais completo e fácil de apresentar.

Quando a exportação termina, o sistema exibe:

Relatorio final gerado com sucesso: relatorio_final_orion_x.xlsx

12. Função main()

Por fim, a função main() organiza a execução geral do programa.

Ela primeiro mostra o título do sistema, depois chama:

exibir_estruturas_missao()

processar_dados_missao()

Isso significa que o programa começa exibindo as estruturas de dados e, em seguida, processa a planilha da missão.

Por fim, imprime:

Processamento concluído com sucesso!

A linha:

```
if __name__ == "__main__":
    main()
```

garante que o programa será executado corretamente quando o arquivo Python for aberto diretamente.

Portanto, o código desenvolvido atende aos requisitos da avaliação porque utiliza estruturas de dados, laços de repetição, leitura e manipulação de Excel, análise com Pandas, criação de novas colunas, filtros condicionais, integração com API Web e geração automática de relatório final.

Além disso, o sistema simula uma situação real da indústria espacial, em que dados precisam ser analisados rapidamente para identificar riscos e apoiar decisões. Dessa forma, a aplicação demonstra como Python pode ser usado para automatizar processos, organizar informações e gerar análises úteis em uma missão espacial fictícia.

a) Estruturas de dados

Na aplicação desenvolvida, foram utilizadas **tuplas, dicionários e laços de repetição** para armazenar e exibir informações da missão espacial fictícia Orion-X.

As **tuplas** foram usadas para armazenar as coordenadas simuladas da nave, contendo latitude, longitude e altitude. Esse tipo de estrutura foi escolhido porque as coordenadas representam um conjunto fixo de informações.

Os **dicionários** foram utilizados para organizar os dados dos sensores e dos módulos da nave. Cada sensor possui informações como nome, tipo, módulo responsável e unidade de medida. Já os módulos armazenam dados de identificação e função dentro da missão.

Os **laços de repetição** foram usados para percorrer essas estruturas e imprimir os dados de maneira organizada no terminal, permitindo visualizar os sensores cadastrados, os módulos da missão e as coordenadas da nave.

Dessa forma, a aplicação consegue representar informações importantes da missão, como sensores, coordenadas, status da nave, temperaturas e identificação dos módulos.

b) Manipulação de arquivos Excel



base_missao_orion_
x.xlsx



relatorio_final_orio
n_x.xlsx

A manipulação de arquivos Excel foi realizada utilizando a biblioteca **Pandas**, que permite ler, tratar e salvar arquivos no formato .xlsx.

O sistema lê a base fictícia da missão a partir do arquivo base_missao_orion_x.xlsx. Essa planilha contém dados simulados, como temperatura, energia, oxigênio, radiação, comunicação e status dos módulos da nave.

Após a leitura da planilha, o programa manipula os dados, cria novas informações e organiza os registros analisados. Entre as novas informações adicionadas estão a classificação de risco, a análise automática, o percentual de energia consumida e o índice de estabilidade da missão.

Ao final do processamento, o sistema salva um novo arquivo chamado relatorio_final_orion_x.xlsx, que funciona como relatório automatizado da missão. Esse relatório contém os dados analisados e organizados em diferentes abas, facilitando a interpretação das informações monitoradas.

c) Análise de dados com pandas

A análise dos dados foi feita com a biblioteca **Pandas**, utilizando os principais comandos solicitados na avaliação.

O comando `.head()` foi utilizado para exibir as primeiras linhas da base de dados, permitindo uma visualização inicial dos registros. O comando `.tail()` mostrou as últimas linhas da planilha. Já o `.info()` apresentou informações gerais sobre as colunas, tipos de dados e possíveis valores nulos. O `.describe()` gerou um resumo estatístico dos dados numéricos, como média, valor mínimo, valor máximo e desvio padrão.

Além disso, foram aplicados **filtros condicionais** para identificar registros em situação de atenção ou crítica. Por exemplo, o sistema separa automaticamente os registros em que a temperatura está elevada, a energia está baixa, o oxigênio está reduzido, a radiação está alta ou a comunicação está instável.

Também foram criadas novas colunas, como:

- `classificacao_risco`;
- `analise_automatica`;
- `energia_consumida_pct`;
- `indice_estabilidade`.

A coluna `classificacao_risco` classifica cada registro como Normal, Atenção ou Crítico. A coluna `analise_automatica` gera uma explicação textual sobre os problemas identificados. O resumo estatístico permite analisar o comportamento geral dos dados monitorados durante a missão.

d) Integração com API Web

A integração com API Web foi realizada utilizando a biblioteca **requests**. O sistema consome uma API pública de clima, simulando a coleta de dados externos relacionados ao ambiente de apoio da missão.

A API retorna informações em formato **JSON**, que são lidas e tratadas pelo programa. Entre os dados coletados estão temperatura externa, velocidade do vento, direção do vento e horário da coleta.

O código também utiliza tratamento de erros com **try/except**, garantindo que o sistema não pare de funcionar caso ocorra falha de conexão ou erro na resposta da API. Se a coleta for realizada com sucesso, os dados são exibidos no terminal e incluídos no relatório final. Caso ocorra erro, o programa informa o problema e continua a execução.

Essa etapa demonstra como uma aplicação Python pode se conectar a serviços externos e utilizar dados em tempo real para complementar a análise da missão.

e) Automação e relatório final



A aplicação automatiza todo o processamento dos dados da missão espacial. Primeiro, o sistema lê a base fictícia em Excel. Depois, realiza a análise dos dados, cria novas colunas, classifica os registros, gera alertas e consolida as informações em resumos.

O relatório final é exportado automaticamente para o arquivo `relatorio_final_orion_x.xlsx`. Esse arquivo contém várias abas organizadas, como:

- dados analisados;
- resumo estatístico;
- resumo por classificação;
- alertas críticos;
- alertas de atenção;
- dados da API;
- resumo geral da missão.

Essa organização facilita a consulta das informações e permite identificar rapidamente a situação da missão. Com isso, o sistema atende ao requisito de automação, pois realiza a consolidação, análise e geração do relatório final sem necessidade de edição manual.

Portanto, a aplicação desenvolvida demonstra o uso prático de Python, Pandas, Excel, API Web e estruturas de dados para simular um sistema de monitoramento inteligente aplicado à indústria espacial.

• Conclusão

O desenvolvimento do Sistema de Monitoramento da Missão Espacial Orion-X permitiu aplicar, de forma prática e integrada, os principais conceitos estudados ao longo da disciplina. Através da utilização da linguagem Python, foi possível criar uma solução capaz de armazenar, processar, analisar e organizar dados simulados de uma missão espacial, demonstrando como a programação pode ser utilizada para resolver problemas relacionados ao monitoramento e à tomada de decisões em ambientes complexos.

Durante a construção do projeto, foram utilizados recursos importantes como tuplas, dicionários e laços de repetição para estruturar as informações da missão, além da biblioteca Pandas para leitura, manipulação e análise de dados em arquivos Excel. Também foi realizada a integração com uma API Web, permitindo a obtenção de informações externas em formato JSON e demonstrando a comunicação da aplicação com serviços disponíveis na internet. Por fim, todo o processamento foi automatizado por meio da geração de relatórios organizados e exportados para Excel.

Além dos aspectos técnicos, o projeto proporcionou uma compreensão mais ampla sobre a importância da análise de dados e da automação na indústria espacial. Foi possível perceber como informações coletadas por sensores podem ser transformadas em conhecimento útil para identificar riscos, monitorar o desempenho dos sistemas e apoiar decisões estratégicas durante uma missão.

Como aprendizado, o trabalho contribuiu para o desenvolvimento do raciocínio lógico, da capacidade de manipulação de dados e da integração entre diferentes tecnologias estudadas na disciplina. Dessa forma, o projeto atingiu seus objetivos e demonstrou a relevância da programação como ferramenta fundamental para o monitoramento inteligente e a gestão de informações em cenários inspirados nos desafios reais da exploração espacial.

Arquivos utilizados e link do código:

https://colab.research.google.com/drive/1wTLHYjENj5p81a_cfKO8a1lu6dVDBZUi?usp=drive_link



Link do vídeo pitch postado no youtube:

https://youtu.be/SdGJy-tH3mg?si=R_laEYavc0jXfas

